

Smart Gym AI System

UML Diagrams — Database model

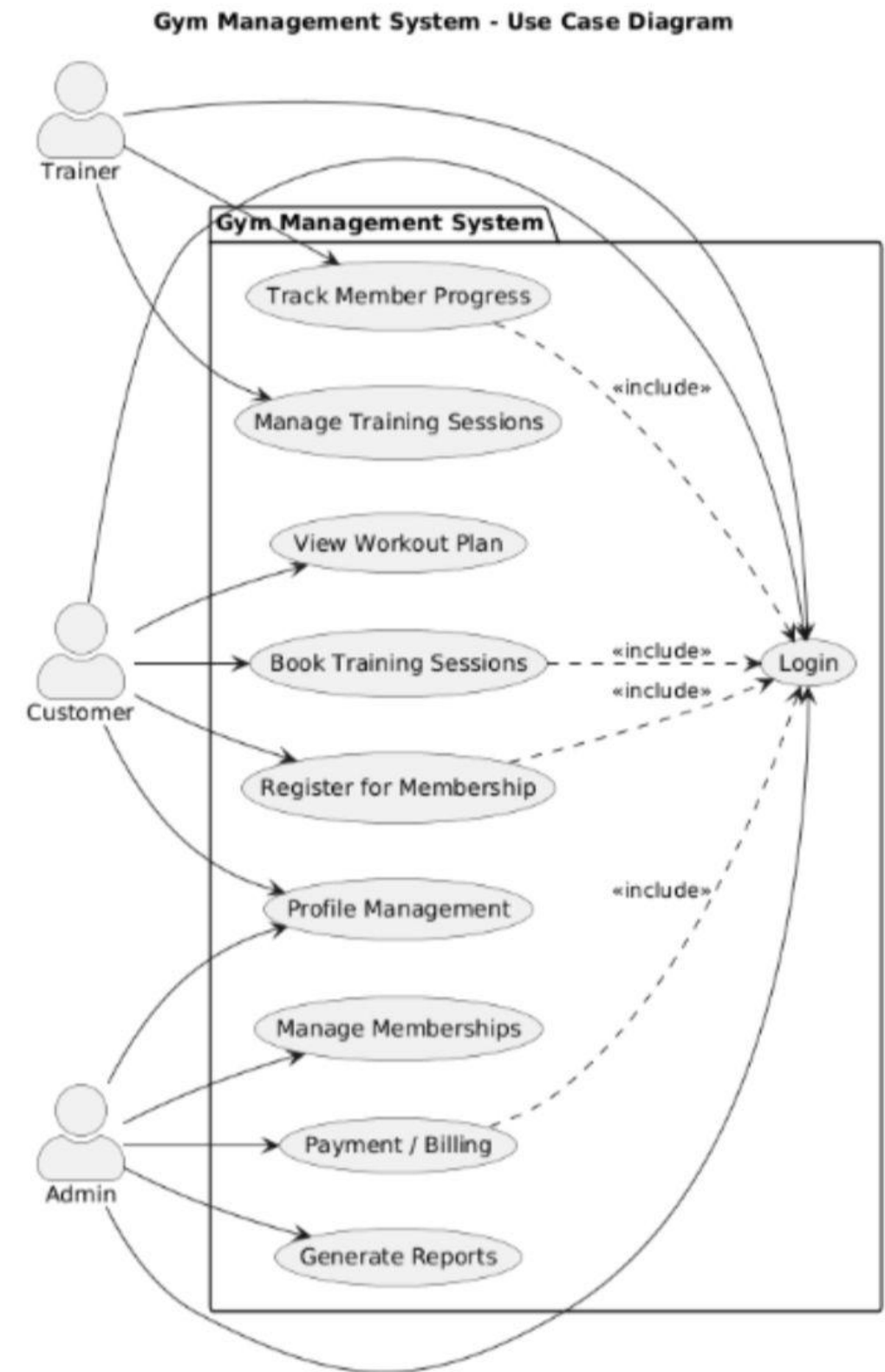
Supervisor: Dr. Mohammed El-Toukhy

Student name: Faisal alshammari , Mohammed Almutairi , Aseel aldrieweesh , Mohammed aldrieweesh ,Omar aljarbaa



Gym Management System

Use Case Diagram — Core actor interactions



Three Actors, One System

Trainer

- Track Member Progress ▪
- Manage Training Sessions ▪
- Login ▪

Customer

- View Workout Plan ▪
- Book Sessions ▪
- Register Membership ▪
- Profile Management ▪
- Login ▪

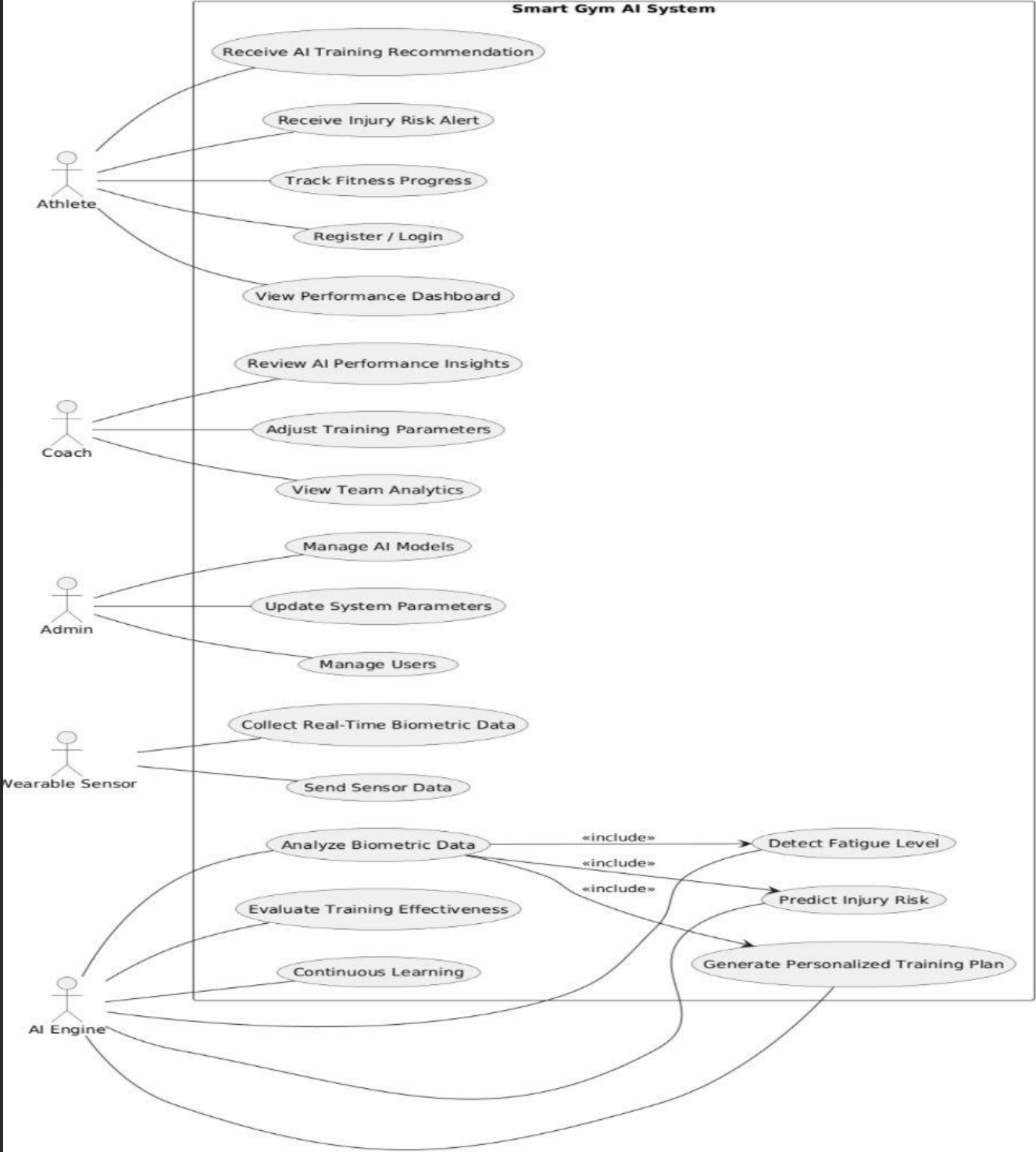
Admin

- Manage Memberships ▪
- Payment / Billing ▪
- Generate Reports ▪
- Login ▪

📌 Login is included by Track Progress, Book Sessions, Register, Profile Management, and Payment/Billing.

Smart Gym AI – Use Case Overview

5 actors: Athlete, Coach, Admin, Wearable Sensor, AI Engine



AI System Actors & Use Cases



Athlete

- AI Training Recommendation
- Injury Risk Alert
- Track Fitness Progress
- Performance Dashboard



Coach

- AI Performance Insights
- Adjust Training Parameters
- Team Analytics



Admin

- Manage AI Models
- Update System Parameters
- Manage Users



Wearable Sensor

- Collect Biometric Data
- Send Sensor Data



AI Engine

- Analyze Biometric Data
- Evaluate Training Effectiveness
- Continuous Learning

AI Engine — Core Analysis

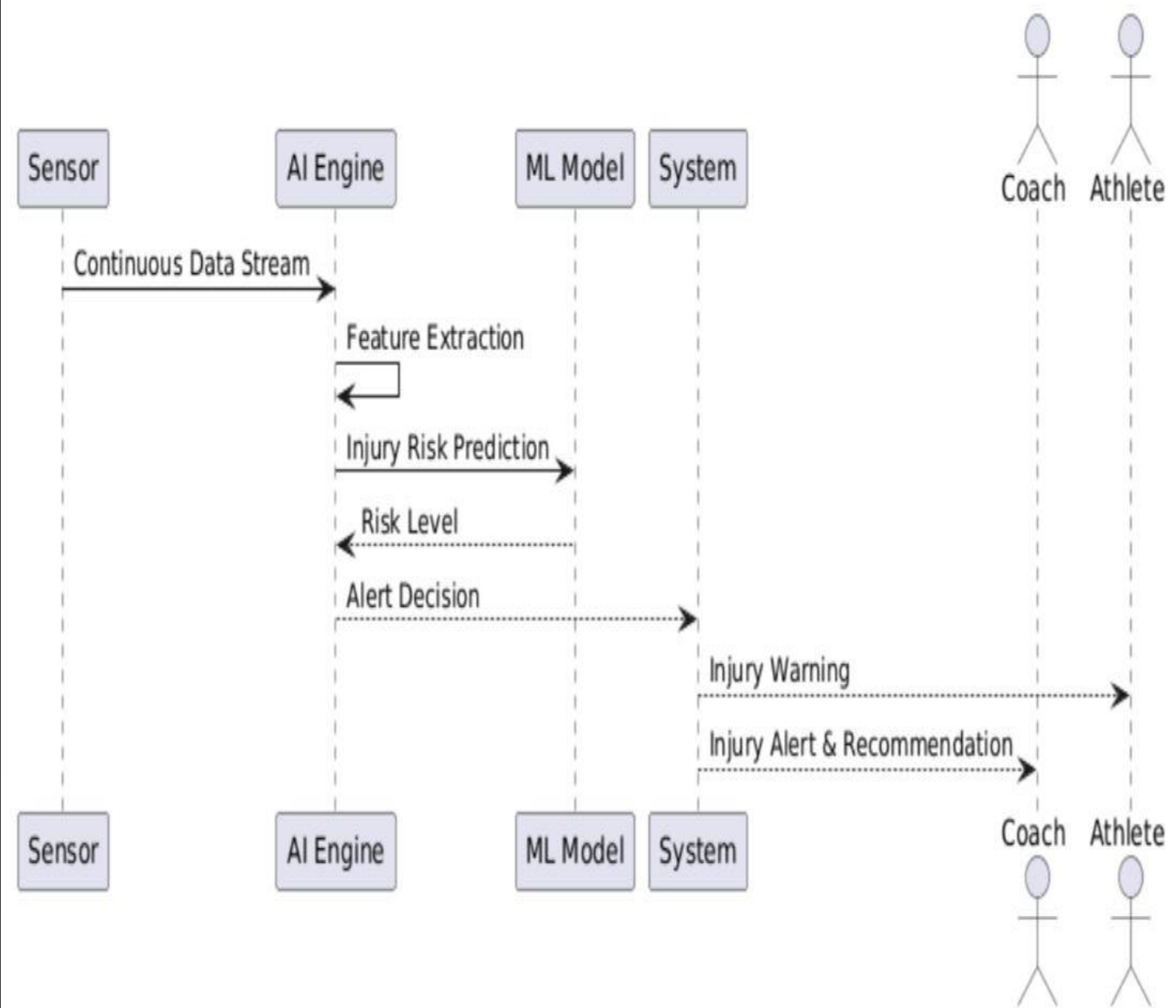


Analyze Biometric Data includes:

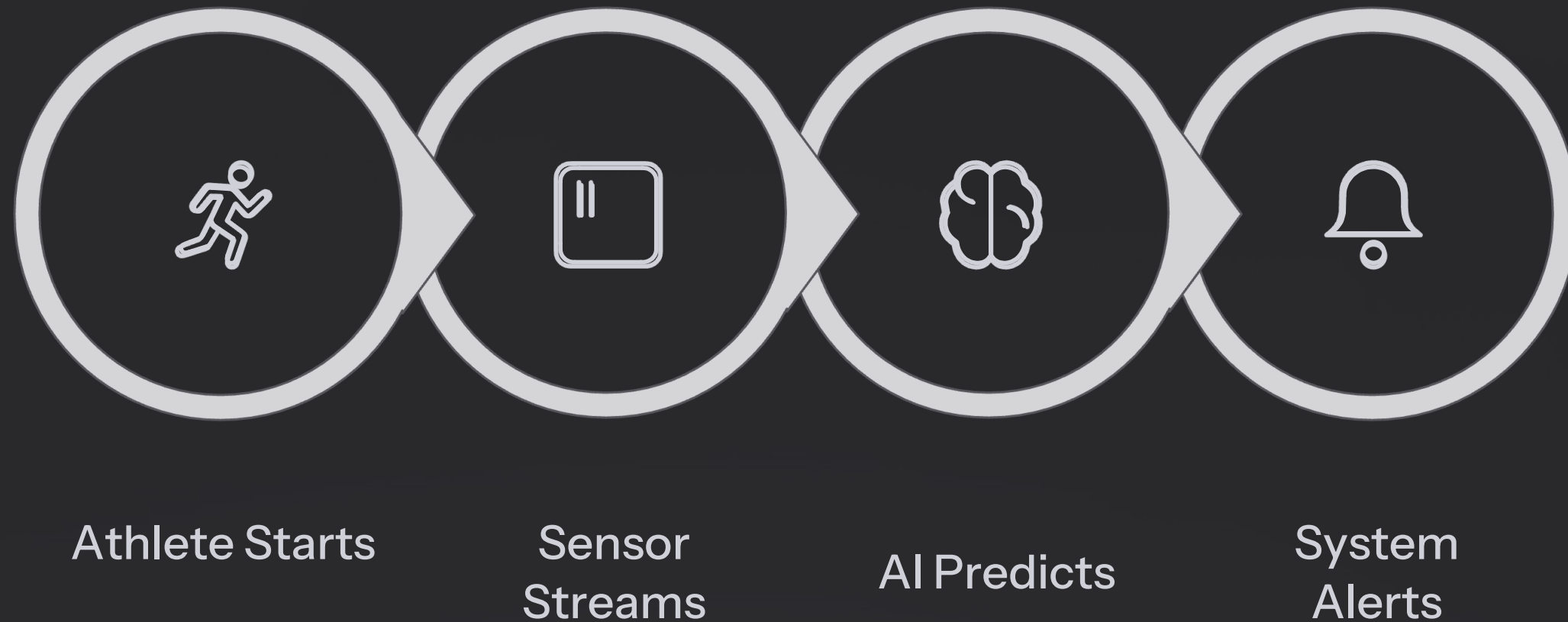
- Detect Fatigue Level
- Predict Injury Risk
- Generate Personalized Training Plan

Injury Prediction — Sequence Flow

Sensor → AI Engine → ML Model → System → Athlete & Coach



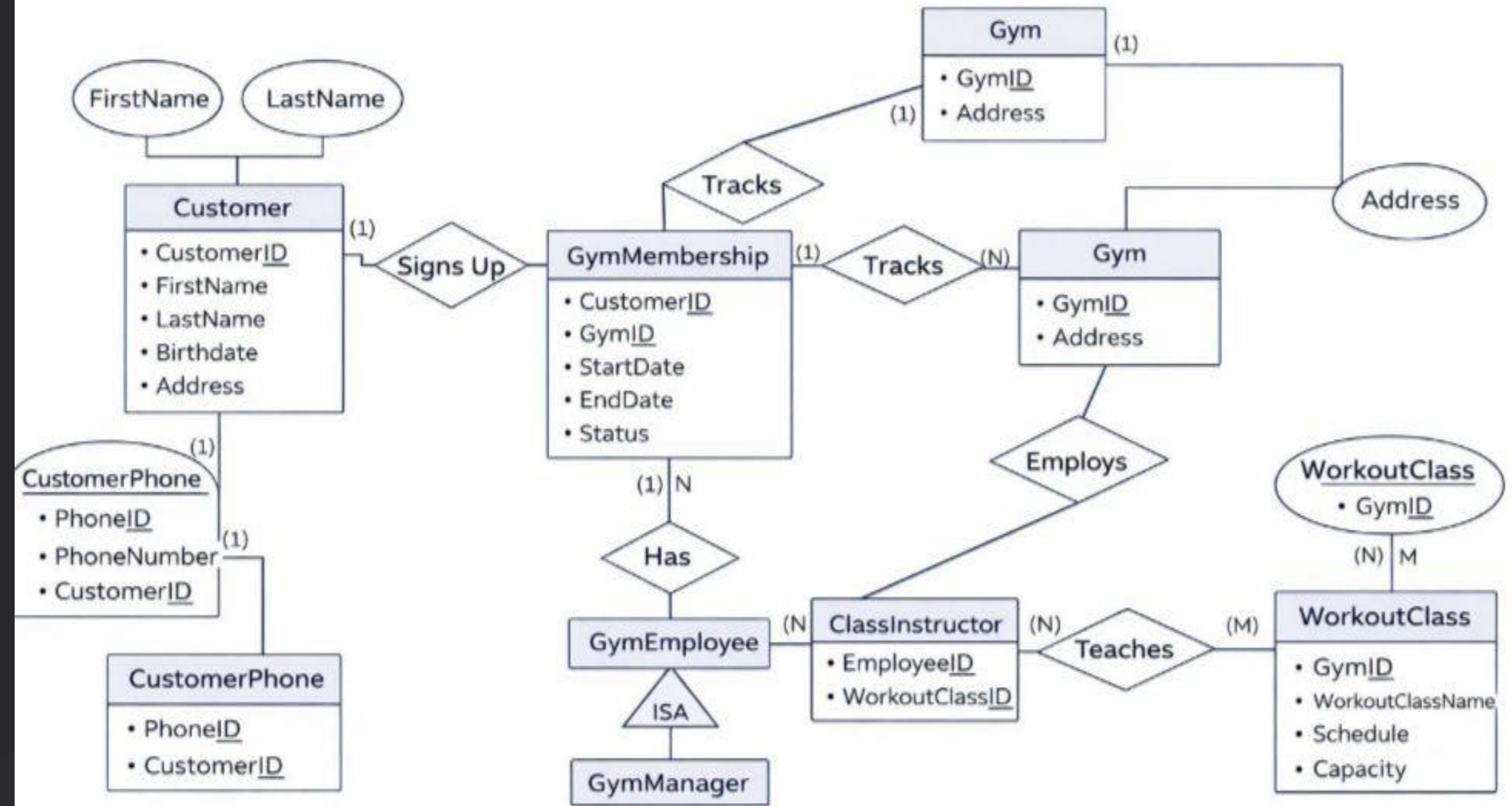
Training Session — Sequence Flow



Athlete initiates → Sensor streams data → AI Engine + ML Model predict fatigue & injury → System delivers personalized plan & alerts.

ER Diagram

This the RE Diagram:



Database model:

This image shows a **database schema** for a health and fitness tracking system .

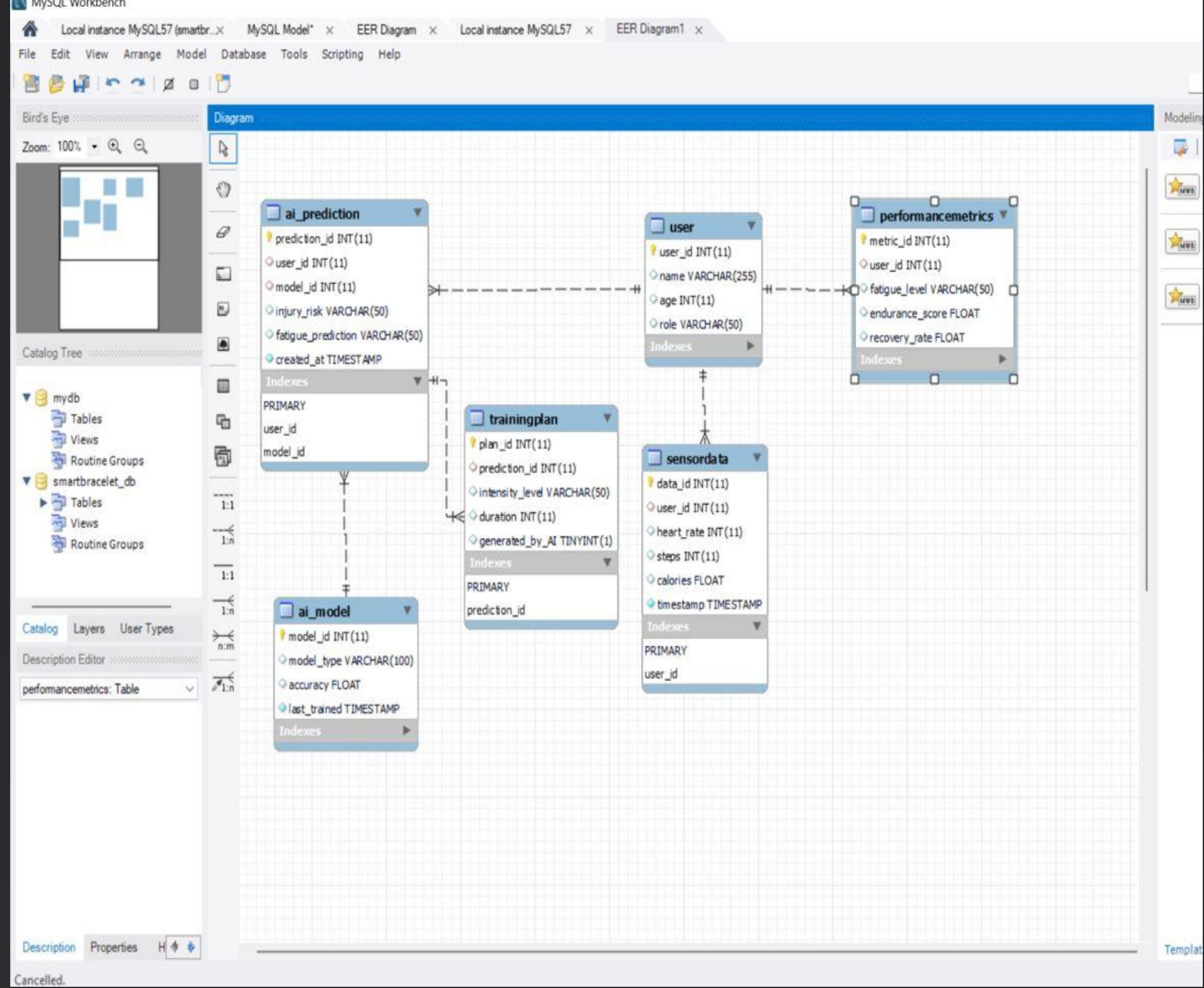
- User Management:** Stores personal details like name and age.

- Health Tracking:** The **sensordata** table collects real-time info like heart rate and steps from the smart bracelet.

- AI Integration:** The system uses **AI models** to generate predictions about injury risks and fatigue levels.

- Smart Training:** Based on these AI predictions, the system creates a personalized **trainingplan** for the user.

- Performance Monitoring:** It tracks long-term metrics like **endurance scores** and **recovery rates** to see how the user is progressing.



The code of database model

This the code of database model:

Limit to 1000 rows

```
1  /* -----
2  DATABASE SCHEMA: SMART BRACELET FOR STRESS MONITORING
3  Purpose: Storing physiological data, AI predictions, and training plans.
4  -----
5  */
6
7  -- Create and initialize the database schema
8  • DROP DATABASE IF EXISTS smartbracelet_db;
9  • CREATE DATABASE IF NOT EXISTS smartbracelet_db;
10 • USE smartbracelet_db;
11
12 -- 1. USER PROFILE TABLE
13 -- Stores personal and demographic information of the system's users.
14 • CREATE TABLE User (
15     user_id INT PRIMARY KEY,
16     name VARCHAR(255),
17     age INT,
18     role VARCHAR(50)
19 );
20
21 -- 2. AI MODELS REGISTRY
22 -- Contains metadata and performance metrics for the stress-detection models.
23 • CREATE TABLE AI_Model (
24     model_id INT PRIMARY KEY,
25     model_type VARCHAR(100),
26     accuracy FLOAT,
27     last_trained TIMESTAMP
28 );
```

```
29
30 -- 3. SENSOR DATA LOGS
31 -- Captures real-time physiological and activity data from the wearable hardware.
32 • CREATE TABLE SensorData (
33     data_id INT PRIMARY KEY,
34     user_id INT,
35     heart_rate INT,
36     steps INT,
37     calories FLOAT,
38     timestamp TIMESTAMP,
39     FOREIGN KEY (user_id) REFERENCES User(user_id)
40 );
41
42 -- 4. PHYSICAL PERFORMANCE METRICS
43 -- Tracks derived physiological states such as fatigue levels and recovery rates.
44 • CREATE TABLE PerformanceMetrics (
45     metric_id INT PRIMARY KEY,
46     user_id INT,
47     fatigue_level VARCHAR(50),
48     endurance_score FLOAT,
49     recovery_rate FLOAT,
50     FOREIGN KEY (user_id) REFERENCES User(user_id)
51 );
52
53 -- 5. AI ANALYTICS & PREDICTIONS
54 -- Logs the diagnostic inferences generated by the AI model regarding health risks.
55 • CREATE TABLE AI_Prediction (
56     prediction_id INT PRIMARY KEY,
57     user_id INT,
58     model_id INT,
59     injury_risk VARCHAR(50),
60     fatigue_prediction VARCHAR(50),
61     created_at TIMESTAMP,
62     FOREIGN KEY (user_id) REFERENCES User(user_id),
63     FOREIGN KEY (model_id) REFERENCES AI_Model(model_id)
64 );
65
66 -- 6. AI-GENERATED TRAINING PLANS
67 -- Stores personalized workout recommendations based on individual stress predictions.
68 • CREATE TABLE TrainingPlan (
69     plan_id INT PRIMARY KEY,
70     prediction_id INT,
71     intensity_level VARCHAR(50),
72     duration INT,
73     generated_by_AI BOOLEAN,
74     FOREIGN KEY (prediction_id) REFERENCES AI_Prediction(prediction_id)
75 );
76
```

Challenges:

1-Massive Data Volume (Scalability)

- The **sensordata** table stores frequent inputs like **heart_rate**, **steps** and **calories**.
- Since smart bracelets often sync data every few seconds, this table will grow extremely large very quickly.
- Without an archiving strategy, simple queries to retrieve a user's history will become very slow over time.

2- .Missing Time Context in Metrics

- The **performancemetrics** table tracks critical data like **endurance_score** and **recovery_rate**.
- However, unlike the **sensordata** or **ai_prediction** tables, it lacks a **timestamp** column.
- Without a recorded time, you cannot track a user's progress or create "improvement graphs" because you won't know when a specific score was achieved.

3- .Data Redundancy & Conflicts

- There is a **fatigue_prediction** field in the **ai_prediction** table and a **fatigue_level** field in the **performancemetrics** table.
- Storing similar information in two different places can lead to "data inconsistency"—where the AI predicts one thing, but the recorded metric says another—making it hard to trust the data.

•4- .Rigid Relationship for Training Plans

- The **trainingplan** table is directly linked to **prediction_id**.
- This means a training plan **cannot** exist unless there is an AI prediction first.
- If you wanted to offer a "Standard Plan" or a "Manual Plan" that doesn't rely on AI, this database structure would currently block that.

5-Data Integrity Issues

- Several columns like **injury_risk**, **fatigue_prediction** and **intensity_level** use the **VARCHAR** type.
- Using plain text allows for inconsistent entries (e.g., one system might write "High," another "high," and another "HIGH").
- This makes filtering and analyzing data difficult compared to using standardized ID numbers or specific categories.